

Zadanie 1

Napisz program, który symuluje grę papier–kamień–nożyce pomiędzy użytkownikiem a komputerem. Użytkownik wprowadza "r" (rock, kamień), "p" (paper, papier), albo "s" (scissors, nożyce). Wprowadzenie innej danej jest sygnalizowane jako błąd, ale program kontynuuje działanie aż do przeprowadzenia z góry zadanej w programie liczby rund.

Komputer losuje swój ruch za pomocą generatora liczb losowych.

Uwaga: Aby uzyskać losową liczbę całkowitą z *domkniętego* przedziału $[a, b]$, można użyć

```
from random import randint
...
r = randint(a, b)
```

Po każdej rundzie drukowany jest jej rezultat, a po zakończeniu wszystkich rund, podsumowanie rezultatów.

Przebieg gry dla zadanej liczby rund 5 mógłby zatem wyglądać tak

```
Your move ([r]ock, [p]aper, [s]cissors) -> s
scissors paper: winner: user
Your move ([r]ock, [p]aper, [s]cissors) -> p
paper scissors: winner: comp
Your move ([r]ock, [p]aper, [s]cissors) -> r
rock rock: winner: draw
Your move ([r]ock, [p]aper, [s]cissors) -> z
*** Enter "r" or "p" or "s". Try again...
Your move ([r]ock, [p]aper, [s]cissors) -> r
rock paper: winner: comp
Your move ([r]ock, [p]aper, [s]cissors) -> s
scissors paper: winner: user
```

S U M M A R Y

user	comp	winner
scissors	paper	user
paper	scissors	comp
rock	rock	draw
rock	paper	comp
scissors	paper	user

The user won 2 games, the computer 2, and there were 1 draws

Zadanie 2

Napisz funkcję **best3** która mając dany strumień krotek (**name**, **score**) (reprezentowany przez listę takich krotek przesłaną jako argument) pamięta cały czas trzech liderów (osoby i ich najlepsze wyniki) i na koniec zwraca trzech zwycięzców. Począwszy od momentu gdy liczba różnych osób (reprezentowanych przez ich imiona) osiągnie trzy, drukowana jest informacja o każdej zmianie na pozycji trzech liderów. Na przykład dla listy

```
lst = [ ('a', 14), ('b', 17), ('a', 15), ('b', 7),
        ('c', 16), ('e', 10), ('f', 14), ('e', 17),
        ('a', 13), ('c', 19), ('d', 15), ('b', 18),
        ('d', 11), ('a', 12), ('d', 10), ('d', 10) ]
```

wydruk powinien być

```
leaders changed: [('b', 17), ('c', 16), ('a', 15)]
leaders changed: [('b', 17), ('e', 17), ('c', 16)]
leaders changed: [('c', 19), ('b', 17), ('e', 17)]
leaders changed: [('c', 19), ('b', 18), ('e', 17)]

Final winners: [('c', 19), ('b', 18), ('e', 17)]
```

Zadanie 3

Napisz dekorator **withDebug** taki, że udekorowana nim dowolna funkcja drukuje informację o każdym wywołaniu (przekazane argumenty, zwracana wartość, być może stempel czasowy) wtedy i tylko wtedy, gdy przy wywołaniu dodany jest nazwany argument (*keyword argument*) o nazwie 'DEBUG' i dowolnej wartości (oczywiście ten argument powinien być usunięty przy wywoływaniu oryginalnej funkcji).

Na przykład następujący program

```
import functools, time # time.time() gives a time stamp download DecoDebug.py

def withDebug(func):
    #
    # ...
    #

@withDebug
def adder(a, b):
    return a + b

print('with debug:', adder(1, 2, DEBUG='Yes'))
print(' no debug:', adder(3, 4))
print('with debug:', adder(5, 6, DEBUG=None))
```

powinien wydrukować coś w rodzaju

```

*** 1775737502.333: calling adder
    args=(1, 2) kwargs={}
    Result returned=3
with debug: 3
    no debug: 7
*** 1775737502.333: calling adder
    args=(5, 6) kwargs={}
    Result returned=11
with debug: 11

```

Zadanie 4

Napisz fabrykę dekoratorów **deprecated**, która przyjmuje jeden argument – nazwę pewnej funkcji. Jeśli funkcja udekorowana tym dekoratorem jest wywoływana, powinna działać normalnie, ale program powinien wypisać komunikat, że dana funkcja jest przestarzała i podać nazwę zamiennika, która była przekazana jako argument do dekoratora.

Na przykład następujący program

```

@deprecated('goodAdd')
def badAdd(a, b):
    return a + b

@deprecated('awesomeFun')
def patheticFun(*, a, b):
    return a * b

c = badAdd(1, 7)
print(f'{c=}')

d = patheticFun(a=3, b=7)
print(f'{d=}')

```

powinien wypisać

```

!!! badAdd is deprecated, use goodAdd instead !!!
c=8
!!! patheticFun is deprecated, use awesomeFun instead !!!
d=21

```

Zadanie 5

Dana jest macierz liczb *m* (lista list). Stwórz listę składaną 2-krotek zawierających indeks wiersza macierzy i średnią arytmetyczną elementów tego wiersza, ale tylko dla takich wierszy dla których ta średnia jest większa lub równa zadanej wartości zmiennej *minave*.

Uwaga: średnia dla każdego wiersza może być wyliczana *tylko raz!*

Na przykład program

```

m = [ [10, 20, 3],
      [ 2, 3, 4, 5, 6],
      [20, 20, 25, 0, -15] ]
minave = 10

lst = [ <list comprehension> ]
print(lst)

```

powinien wydrukować [(0, 11.0), (2, 10.0)].

Zadanie 6

Wszystkie poniższe zadania należy rozwiązać za pomocą składanych list, słowników, i/lub zbiorów. **W każdym powinny wystarczyć trzy linijki kodu:**

1. Definicja danych (lista, słownik).
2. Użycie składania i przypisanie wyniku do zmiennej.
3. Drukowanie wyniku.

- **1.** Mając listę `lst` elementów różnych typów, utwórz listę nazw typów kolejnych elementów

Na przykład dla `lst = [2, {}, 2.3, 'abc', (4,), 2+2j]`, wynik powinien być

```
['int', 'dict', 'float', 'str', 'tuple', 'complex']
```

- **2.** Mając daną listę `lst` list liczb całkowitych utwórz listę wszystkich występujących tam liczb, ale tylko z zakresu `[0, 10]`.

Na przykład dla `lst = [[1, 11], [7, -3, 2], [5, 12, -6]]`, wynik powinien być

```
[1, 7, 2, 5]
```

- **3.** Mając daną listę `lst` list liczb całkowitych utwórz listę wszystkich występujących tam liczb, ale zastępując liczby ujemne zerem, a te większe od 10 liczbą 10.

Na przykład dla `lst = [-2, 9, 11, 7, 1, 12, 9]` wynik powinien być

```
[0, 9, 10, 7, 1, 10, 9]
```

- **4.** Mając daną listę `lst` referencji do niemodyfikowalnych elementów, stwórz słownik gdzie kluczami będą te elementy, a wartościami liczby ich wystąpień (za pomocą słownika składanego, *dictionary comprehension*).

Uwaga: Przyda się metoda **count** listy: `aList.count(v)` daje liczbę wystąpień wartości `v` w liście.

Na przykład dla listy

```
lst = ['Eve', 'Jane', 'Eve', 'Mary',  
       'John', 'Mary', 'Eve', 'John']
```

wynik powinien być {'Jane': 1, 'John': 2, 'Eve': 3, 'Mary': 2}

- 5. Mając daną prostokątną macierz elementów (czyli listę list o tej samej długości), utwórz macierz transponowaną, czyli listę list w której kolejne wiersze są takie jak kolejne kolumny oryginalnej macierzy.

Na przykład dla macierzy

```
lst = [  
    [1, 2, 3, 4],  
    [5, 6, 7, 8],  
    [1, 1, 1, 2]  
]
```

wynik powinien być [[1, 5, 1], [2, 6, 1], [3, 7, 1], [4, 8, 2]]

- 6. Dana jest lista temperatur (w stopniach Celsjusza) płynu w chłodnicy samochodu. Utwórz listę kolorów lampki ostrzegawczej według zasady: temperatura mniejsza niż 100 ⇒ zielony, między 100 a 115 ⇒ żółty, ponad 115 ⇒ czerwony.

Na przykład dla listy

```
lst = [ 87, 102, 90, 117, 95 ]
```

wynik powinien być ['green', 'yellow', 'green', 'red', 'green']

- 7. Dana jest lista nazw plików z różnymi rozszerzeniami. Utwórz słownik, w którym kluczami są rozszerzenia, a wartościami listy plików z tym rozszerzeniem. Możesz założyć, że wszystkie pliki mają rozszerzenia (część nazwy po ostatniej kropce).

Może się przydać metoda **endwith** klasy **str**.

Na przykład dla listy

```
lst = ['a.pdf', 'b.c', 'c.pdf', 'd.java', 'e.f.pdf', 'g.c']
```

wynik powinien być (kolejność elementów może być inna)

```
{'c': ['b.c', 'g.c'], 'java': ['d.java'],  
 'pdf': ['a.pdf', 'c.pdf', 'e.f.pdf']}
```

- 8. Dana jest trzypoziomowa lista (lista list list). Utwórz zwykłą, jednopoziomową listę zawierającą wszystkie jej elementy.

Na przykład dla listy

```
lst = [ [[1, 2], ['a']], [[3, 4, 5], ['b']] ]
```

wynik powinien być [1, 2, 'a', 3, 4, 5, 'b']

● **9.** Dana jest lista krotek, z których każda zawiera nazwę produktu i cenę za jaką go sprzedano. Utwórz słownik, w którym kluczami są nazwy produktów, a wartościami listy wszystkich cen za jakie dany produkt sprzedano. Na podstawie tej samej listy krotek utwórz słownik, w którym kluczami są nazwy produktów, a wartościami całkowite zarobione pieniądze za każdy produkt.

Na przykład dla listy

```
lst = [ ('p1', 12), ('p2', 17), ('p1', 13),  
        ('p3', 20), ('p2', 11), ('p1', 19) ]
```

wynikiem powinny być słowniki

```
{ 'p2': [17, 11], 'p3': [20], 'p1': [12, 13, 19] }
```

oraz

```
{ 'p2': 28, 'p3': 20, 'p1': 44 }
```

● **10.** Dana jest lista haseł. Utwórz listę 2-krotek w których pierwszym elementem jest hasło, a drugim wartość **True** lub **False** w zależności od tego, czy hasło spełnia następujące wymogi:

1. Zawiera co najmniej 8 znaków.
2. Zawiera co najmniej jedną cyfrę.
3. Zawiera co najmniej jedną literę.
4. Zawiera co najmniej jeden znak, który nie jest ani literą ani cyfrą.

Z nieznanych przyczyn lista może zawierać elementy **None** lub takie, które nie są napisami — powinny one być pominięte.

Przydatne mogą być funkcje **any** i **all**, jak również metody klasy **str** **isalpha** i **isdigit**.

Na przykład dla listy

```
passes = ['Bond@007', 'Abracadabra!', None,  
          'Top/Secret', 12.3, '<abc123>', 'abc123z9']
```

wynikiem powinna być lista

```
[('Bond@007', True), ('Abracadabra!', False),  
 ('Top/Secret', False), ('<abc123>', True),  
 ('abc123z9', False)]
```

● **11.** Mając dany słownik, w którym kluczami są nazwy modeli samochodów, a wartościami nazwy ich marki, utwórz słownik w którym kluczami są marki a wartościami listy modeli.

Na przykład dla słownika

```
dct = { 'Astra': 'Opel', 'RAV4': 'Toyota',
        'Corsa': 'Opel', 'Clio': 'Renault',
        'Prius': 'Toyota', 'Vectra': 'Opel',
        'Camry': 'Toyota', 'Megane': 'Renault' }
```

wynik powinien być

```
{'Opel': ['Astra', 'Corsa', 'Vectra'],
 'Toyota': ['RAV4', 'Prius', 'Camry'],
 'Renault': ['Clio', 'Megane']}
```

● 12. Dany jest słownik, w którym kluczami są imiona studentów, a wartościami słowniki ich ocen z testów z różnych przedmiotów. Wyniki są podane w punktach na 20 możliwych. Utwórz podobny słownik, w którym oceny są podane w procentach w stosunku do 100.

Na przykład dla słownika

```
dct = { 'Eve': {'math': 12, 'eng': 9, 'ppy': 14},
        'Sue': {'math': 16, 'eng': 12, 'ppy': 12},
        'Ron': {'math': 19, 'eng': 15, 'ppy': 17} }
```

wynik powinien być

```
{ 'Eve': {'math': 60.0, 'eng': 45.0, 'ppy': 70.0},
  'Sue': {'math': 80.0, 'eng': 60.0, 'ppy': 60.0},
  'Ron': {'math': 95.0, 'eng': 75.0, 'ppy': 85.0} }
```

● 13. Dana jest lista wierszy pewnego tekstu. Utwórz słownik, w którym kluczami są słowa występujące w tym tekście, a wartościami zbiory numerów wierszy (poczynając od jedynki, a nie zera), w których te słowa występują.

Na przykład dla listy

```
text = [ "python and java are cool",
         "some people prefer python",
         "while others prefer c and java" ]
```

wynik powinien być

```
{ 'prefer': {2, 3}, 'java': {1, 3}, 'python': {1, 2},
  'while': {3}, 'are': {1}, 'some': {2}, 'and': {1, 3},
  'others': {3}, 'c': {3}, 'cool': {1}, 'people': {2} }
```

Zadanie 7

Użyj mechanizmu *single dispatch* do zdefiniowania funkcji **formatArg** przyjmującej jeden argument i zwracającej jego reprezentację tekstową w formie zależnej od typu argumentu

- **str** — przekazany napis, ale w cudzysłowach;
- **int** lub **float** — przekazana liczba w postaci napisu;
- **list** lub **tuple** — napis zawierający wszystkie elementy, oddzielone przecinkami i ujęte w nawiasy kwadratowe;
- **dict** — napis zawierający ujęte w nawiasy klamrowe pary klucz:wartość, oddzielone przecinkami;
- **set** — napis zawierający ujęte w nawiasy klamrowe posortowane elementy zbioru oddzielone średnikami (zakładamy, że elementy zbioru są sortowalne)
- dla innych typów funkcja powinna ponieść wyjątek **TypeError** (z odpowiednią informacją).

Na przykład następujący program

[download Dispatch.py](#)

```
def pr(arg):
    s = formatArg(arg)
    print(type(s).__name__ + ': ', s)

pr('Hello')
pr(7)
pr(1.2e+2)
pr([1, 2, 3])
pr((4, 5, 6))
pr(dict(a='Hello', b=123))
pr({'z', 'b', 'c', 'a'})
# pr(7+4j)
```

powinien wypisać coś w rodzaju

```
str: "Hello"
str: 7
str: 120.0
str: [1, 2, 3]
str: [4, 5, 6]
str: {a:Hello, b:123}
str: [a;b;c;z]
```

a po odkomentowaniu ostatniej linii powinniśmy dostać wyjątek z komunikatem

```
TypeError:  Type complex not supported.
```